

【实验报告】

课程名： 计算机组成与体系结构（H）

实验名称： lab2 内存相关指令

学生： 杨箫羽

学号： 24300240069

时间： 2026 年 3 月 30 日

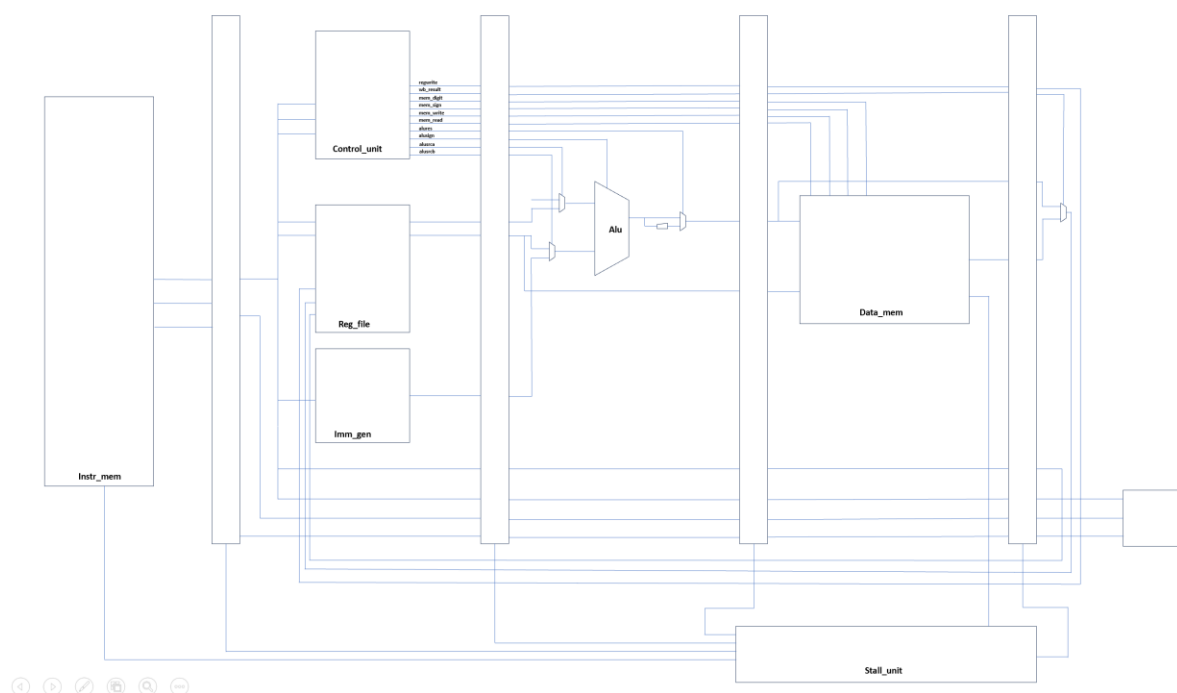
一、实验内容与实现思路说明

实验 2 要求增加和内存存取相关的 11 条指令以及 lui 指令。为了实现这 12 条指令，我在 MEM 阶段增加了 data_mem 模块控制利用 dbus 的内存存取，在 control_unit 中增加了 5 个控制信号，增加了 stall_unit 模块来控制当内存读写未完成时的流水线阻塞。

下面分总体电路图、新增的模块和做出重要修改的模块讲解、调试三部分进行具体说明。

```
Using /root/26-Arch/ready-to-run/riscv64-nemu-interpretor-so for difftest
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'clint' at [0x38000000, 0x3800ffff]
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'uartlite' at [0x40600000, 0x4060000c]
[src/device/io/mmio.c:19,add_mmio_map] Add mmio map 'uartlite1' at [0x23333000, 0x2333300f]
The first instruction of core 0 has committed. Difftest enabled.
[WARNING] difftest store queue overflow
[src/cpu/cpu-exec.c:393,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x000000008001fffc
[src/cpu/cpu-exec.c:394,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:74,monitor_statistic] host time spent = 9802 us
[src/cpu/cpu-exec.c:76,monitor_statistic] total guest instructions = 32767
[src/cpu/cpu-exec.c:77,monitor_statistic] simulation frequency = 3342889 instr/s
```

二、总体电路图



三、修改和新增的重要模块

1、data_mem

接口：

```
module data_mem (
    input logic [63:0] address,
    input logic [63:0] mem_write_data,
    input logic mem_write_m,
    input logic mem_read_m,
    input logic [1:0] mem_digit_m, // 0:8bit, 1:16bit, 2:32bit, 3:64bit
    input logic mem_sign_m, // 1:零扩展, 0:符号扩展

    input dbus_resp_t dresp,
    output dbus_req_t dreq,

    output logic [63:0] mem_read_data,
    output logic mem_stall
);
```

功能：为了描述方便，以理想内存来描述存取。实际实现时，在此功能基础上对应填写 strobe、size 和 addr。

当 mem_write_m 为 1 时，将 mem_write_data 写到 mem[addr]。具体写的规则如下：

如果 mem_digit_m 是 0，那么将 mem_write_data0-7 位的 8 个 bit 数据，写到 mem[address] 中。

如果 mem_digit_m 是 1，那么将 mem_write_data0-15 位的 16 个 bit 数据，写到 mem[address] 和 mem[address+1] 中。

如果 mem_digit_m 是 2，那么将 mem_write_data0-31 位的 32 个 bit 数据，写到 mem[address] 到 mem[address+3] 中。

如果 mem_digit_m 是 3，那么将 mem_write_data0-63 位的 64 个 bit 数据，写到 mem[address] 到 mem[address+7] 中。

在这种情况下，如果 dataok 是 0，那么一直锁住所有流水寄存器和 pc，也就是将 memstall 一直置为 1。

当 mem_read_m 为 1 时，将 mem[address] 的数据存入 rd。具体存放规则如下：

如果 mem_digit_m 是 0，那么读 mem[address] 中 8 个 bit 的数据。在此基础上，如果 mem_sign_m 是 1，零扩展，否则符号扩展到 64 位，写到 mem_read_data

如果 mem_digit_m 是 1，那么读 mem[address] 和 mem[address+1] 中 16 个 bit 的数据。在此基础上，如果 mem_sign_m 是 1，零扩展，否则符号扩展到 64 位，写到 mem_read_data

如果 mem_digit_m 是 2，那么读 mem[address] 到 mem[address+3] 中 32 个 bit 的数据。在此基础上，如果 mem_sign_m 是 1，零扩展，否则符号扩展到 64 位，写到 mem_read_data

如果 mem_digit_m 是 3，那么读 mem[address] 到 mem[address+7] 中 64 个 bit 的数据。在此基础上，如果 mem_sign_m 是 1，零扩展，否则符号扩展到 64 位，写到 mem_read_data

在这种情况下，如果 dataok 是 0，那么一直锁住所有流水寄存器和 pc，也就是将 memstall 一直置为 1。

2、stall_unit

```
module stall_unit (  
    input  logic mem_stall,  
    output logic pc_stall,  
    output logic if_id_stall,  
    output logic id_ex_stall,  
    output logic ex_mem_stall,  
    output logic mem_wb_stall  
);
```

因为担心 dbus 读写内存可能会超过一个时钟周期，因此将 mem_stall 作为输入信号（之一），与 4 个流水寄存器和 IF 阶段的 instr_mem 模块相连。如果没有完成读写内存，那么阻塞时钟。

3、imm_gen：加入 lui 指令之后，立即数的处理变得更加复杂。因此增加了生成立即数的模块，根据 instr 类型分别处理立即数，输出处理好的立即数。

```
module imm_gen (  
    input  logic [31:0] instr_d,  
    output logic [63:0] imm_d  
);
```

4、control_unit

增加了 5 个控制信号，分别控制 alusrcA 接受 0 还是 rs1_val，控制 mem 读使能（mem 写使能上次正好写了），控制 datamem 数据的扩展形式，控制 datamem 数据的读写位数，控制 wb 阶段写回数据的来源。

为了清晰，建立了一个表格来描述所有指令的控制信号。

	addi	xori	ori	andi	add	sub	and	or	xor	addiw	addw	subw
opcode	0010011	0010011	0010011	0010011	0110011	0110011	0110011	0110011	0110011	0011011	0111011	0111011
funct3	000	100	110	111	000	000	111	110	100	000	000	000
funct7	0000000	0100000	0000000	0000000	0000000	0100000	0000000	0000000	0000000	0000000	0000000	0100000
alusign	1时针对rv64i这种指令，做32->64的扩展	0	0	0	0	0	0	0	0	0	1	1
aluctrl	(0-4) +/异或/或/与/-	0	1	2	3	0	4	3	2	1	0	4
alusrcA	0为0，1为rs1val	1	1	1	1	1	1	1	1	1	1	1
alusrcB	0为rs2val，1为imm	1	1	1	1	0	0	0	0	0	1	0
regwrite	0不写，1写	1	1	1	1	1	1	1	1	1	1	1
mem_write	0不写，1写	0	0	0	0	0	0	0	0	0	0	0
mem_read	0不读，1读	0	0	0	0	0	0	0	0	0	0	0
mem_sign	0时符号扩展，1时零扩展	0	0	0	0	0	0	0	0	0	0	0
mem_digit	8/16/32/64	0	0	0	0	0	0	0	0	0	0	0
wb_result	0为alu结果，1为mem读结果	0	0	0	0	0	0	0	0	0	0	0

	lb	lh	lw	ld	lbu	lhu	lwu	sb	sh	sw	sd	lui
opcode	0000011	0000011	0000011	0000011	0000011	0000011	0000011	0100011	0100011	0100011	0100011	0100011
funct3	000	001	010	011	100	101	111	000	001	010	011	000
funct7	0000000	0000000	0000000	0000000	0000000	0000000	0000000	0000000	0000000	0000000	0000000	0000000
alusign	1时针对rv64i这种指令，做32->64的扩展	0	0	0	0	0	0	0	0	0	0	0
aluctrl	(0-4) +/异或/或/与/-	0	0	0	0	0	0	0	0	0	0	0
alusrcA	0为0，1为rs1val	1	1	1	1	1	1	1	1	1	1	0
alusrcB	0为rs2val，1为imm	1	1	1	1	1	1	1	1	1	1	1
regwrite	0不写，1写	1	1	1	1	1	1	0	0	0	0	1
mem_write	0不写，1写	0	0	0	0	0	0	1	1	1	1	0
mem_read	0不读，1读	1	1	1	1	1	1	0	0	0	0	0
mem_sign	0时符号扩展，1时零扩展	0	0	0	0	1	1	0	0	0	0	0
mem_digit	8/16/32/64	0	1	2	3	0	1	2	0	1	2	3
wb_result	0为alu结果，1为mem读结果	1	1	1	1	1	1	0	0	0	0	0

5、alusrcA 复用器和 wb_res 复用器

处理 lui 寄存器的时候，利用 alu 计算 0+imm 来传递立即数。因此需要给 alusrcA 增加一个选择 0 还是 rs1val 的选择器。Wb 阶段写回的数据有可能是 datamem 读的结果，也可能是

alu 运算的结果，这里也要加一个选择器。

四、调试

1、一开始一直不能仿真，显示 datapath 里有一些不符合规范的代码，好像来自测试文件。在 llm 的建议下，通过二分筛查了 datapath 顶部包含的所有模块，最后发现是 imm_gen 顶部的几行注释忘记写//了。

2、pc = 0x0080000014 卡住

```
commit inst [5]: pc 0080000014 inst 400108b7 wen 0 dst 00000011 data 0000000000000000 <-
```

因为尝试了 test-lab1 可以通过，所以是这条 lui 指令的问题。后来检查了 control_unit，是 lui 的 wb_result 控制信号写错了，解决之后就好了。

3、stall 时重复递交已经递交过的指令。

```
commit inst [8]: pc 0080000020 inst f118a223 wen 0 dst 00000004 data 0000000080020004
```

```
commit inst [9]: pc 0080000020 inst f118a223 wen 0 dst 00000004 data 0000000080020004 <-
```

修改了 commit 的逻辑——只有新进入 wb 阶段的时候才将 commit_valid 置 1，提交一次；如果指令被 stall，不重复提交。